GCA - Report 1

Maarten Roedolf

February 2024

1 Array Flip

```cuda
#include <iostream>
#include <cuda_runtime.h>

__global__ void flipArray(int *input, int *output, int size) {
    int tid = blockIdx.x * blockDim.x + threadIdx.x;
    if (tid < size) {
        output[tid] = input[size - tid - 1];
    }
}
```

This is a CUDA kernel function for flipping our input array. It's declared with the __global__ qualifier, telling te compiler it will run on the GPU and be called from the CPU. Each thread in the GPU will execute this function. We have to calculate the threadId with the current blockId and the blockwidth because the threadId we get is its relative position inside the block. The function then flips the array element at index tid and stores the result in the corresponding position of the output array.

```cuda
int main() {
    const int arraySize = 10000;
    const int arrayBytes = arraySize * sizeof(int);

    int *h_inputArray = (int*)malloc(arrayBytes);
    int *h_outputArray = (int*)malloc(arrayBytes);

    for (int i = 0; i < arraySize; ++i) {
        h_inputArray[i] = i;
    }

    int *d_inputArray, *d_outputArray;
    cudaMalloc((void**)&d_inputArray, arrayBytes);
    cudaMalloc((void**)&d_outputArray, arrayBytes);

    cudaMemcpy(d_inputArray, h_inputArray, arrayBytes, cudaMemcpyHostToDevice);

    int blockSize = 256;
    int gridSize = (arraySize + blockSize - 1) / blockSize;
```

In the main function we generate an array of a given size (1000 in our example) and calculate the amount of bytes we would need to allocate for an array of integers of this size and we actually allocate this memory on the cpu by using the malloc function.

A simple test array is then generated using a simple for-loop by assigning the index of each element to its value.

We also have to allocate the memory we would need for the arrays on the gpu by using the cudaMalloc function and copy the input array from the host to the devince by using cudaMemcpy.

```
int blockSize = 256;
int gridSize = (arraySize + blockSize - 1) / blockSize;

flipArray<<<gridSize, blockSize>>>(d_inputArray, d_outputArray, arraySize);

cudaMemcpy(h_outputArray, d_outputArray, arrayBytes, cudaMemcpyDeviceToHost);

std::cout << "Original array:" << std::endl;
for (int i = 0; i < arraySize; ++i) {
    std::cout << h_inputArray[i] << " ";
}
std::cout << std::endl;

std::cout << "Flipped array:" << std::endl;
for (int i = 0; i < arraySize; ++i) {
    std::cout << h_outputArray[i] << " ";
}
std::cout << std::endl;
```

After all this preparation we calculate the size of our block grid using the size of our array to ensure we have enough blocks and threads for our given array and can finally call our kernel function with all of our previously defined variables.

I also added simple print lines to compare the input array to see if the function we've written actually works.

```
free(h_inputArray);
free(h_outputArray);
cudaFree(d_inputArray);
cudaFree(d_outputArray);

return 0;
}
```

We then finally have to free all the memory on both the cpu and the gpu by using the free en cudaFree respectively to prevent data leaks.